

Proto-Cuneiform Preprocessing

27/02/2024

Description

This file enables preprocessing of cuneiform tablet transliterations, these can be downloaded from CDLI (<https://cdli.mpiwg-berlin.mpg.de/search>). The extraction of proto-cuneiform sign codings is done with the sign lists provided by CDLI. In their syntax, repetitions of signs are indicated by a number followed by the sign type in parentheses, e.g. 1(N01), 3(N01), 5(N14), etc. This is here expanded to yield individual sign tokens, e.g. N01 N01 N01, etc. We also include signs which are followed by ‘#’ (broken but identifiable signs), but we exclude signs followed by ‘?’ (questionable).

Session Info

Give the session info (reduced).

```
## [1] "R version 4.3.2 (2023-10-31)"  
## [1] "x86_64-pc-linux-gnu"
```

Load Libraries

```
library(Hmisc)  
  
##  
## Attaching package: 'Hmisc'  
  
## The following objects are masked from 'package:base':  
##  
##     format.pval, units  
  
library(stringr)  
library(readr)
```

Load Transliteration and Sign List

Loading transliterations and sign lists from CDLI. Note that different datasets can be loaded by choosing the period as a character string below.

```
# choose period (UrukIII, UrukIV, UrukV)  
period <- "UrukIII"  
  
# transliteration as text file with line breaks  
translit.file = paste("~/Github/UpperPalCombinatorics/data/CDLI/",
```

```

        paste(period, "/inscriptions.atf", sep = ""),
        sep = "")
textlines <- scan(translit.file, what = "char", quote = "", comment.char = "",
                encoding = "UTF-8", sep = "\n" , skip = 0)
# load transliteration as single character string
textstring <- read_file(translit.file)
# load sign list
signlist.file = paste("~/Github/UpperPalCombinatorics/data/CDLI/",
                    paste(period, "/signlist.txt", sep = ""),
                    sep = "")
signlist <- scan(signlist.file, what = "char", quote = "", comment.char = "",
                encoding = "UTF-8", sep = "\n" , skip = 0)

```

Process Transliteration

Get vector of IDs for tablets.

```

# the ampersand followed by P identifies a single tablet transliteration
tablet.id <- textlines[grep(pattern = "\\&P", textlines)]
# check length
length(tablet.id)

```

```
## [1] 1000
```

```

# remove ampersand
tablet.id <- gsub(pattern = "\\&", "", tablet.id)
# get just the ID
tablet.id <- substr(tablet.id, 1, 7)

```

Extract and concatenate the transliterations.

```

# get chunks of text by ampersand plus "P" (at the beginning of each tablet)
textchunks <- str_split(textstring, pattern = "\\&P")
# unlist and remove first element (empty)
textchunks <- unlist(textchunks)[-1]
# remove the header
textchunks <- gsub("\\=.*qpc", "", textchunks)

# use signlist provided by CDLI to extract only signs
# create a duplicate signlist with `#` following after the signs,
# this means the sign is partly broken but identifiable
signlist.hashtag <- paste0(signlist,'#')
# concatenate the two lists (i.e. vectors)
signlist <- c(signlist, signlist.hashtag)
# first some characters in the sign list need to be escaped
signlist <- escapeRegex(signlist)
# add white spaces after character strings in signlist
# this helps with only extracting full string matches
signlist <- paste0(signlist,' ')
# remove 'X' as this is a placeholder for unidentified signs
signlist <- signlist[!signlist == 'X ']

# run for loop to extract signs based on sign list
# initialize vectors

```

```

signs.orig.vec <- c()
signs.expand.vec <- c()
for (i in 1:length(textchunks)){
  # note that the order of signs is here sometimes changed,
  # as it follows the order of signs in sign list
  signs.orig <- unlist(str_extract_all(textchunks[i], paste(signlist, collapse = "|")))
  signs <- unlist(str_extract_all(textchunks[i], paste(signlist, collapse = "|")))
  # trim white spaces around signs
  signs.orig <- trimws(signs.orig)
  signs <- trimws(signs)
  # remove the hashtag
  signs <- gsub("#", "", signs)
  # unravel the multiplication syntax to get multiple sign tokens
  try({ # if the processing fails for a certain row, there will be no output for this row,
    # but the try() function allows the loop to keep running
    for (j in 1:length(signs)){
      sign.syntax <- signs[j]
      # get only number
      remainder <- sub(pattern = "\\(.*\\)", "", sign.syntax)
      # if number then multiply with sign type, else
      # leave sign syntax as is
      if (!grepl("[^0-9]", remainder)){
        # get sign.type inside round brackets
        sign.type <- gsub(".*\\((.*)\\).*", "\\1", sign.syntax)
        # repeat given the number ("remainder" from above)
        sign.tokens <- rep(sign.type, remainder)
        # collapse into character string
        sign.tokens <- paste(sign.tokens, collapse = " ")
      } else {
        sign.tokens <- sign.syntax
      }
      signs[j] <- sign.tokens
    }) # end of try() function
    # collapse the signs into one string
    signs <- paste(signs, collapse = " _ ")
    signs.orig <- paste(signs.orig, collapse = " _ ")
    signs.orig.vec <- c(signs.orig.vec, signs.orig)
    signs.expand.vec <- c(signs.expand.vec, signs)
  }
}

```

```

## Error in rep(sign.type, remainder) : invalid 'times' argument
## Error in rep(sign.type, remainder) : invalid 'times' argument
## Error in rep(sign.type, remainder) : invalid 'times' argument
## Error in rep(sign.type, remainder) : invalid 'times' argument
## Error in rep(sign.type, remainder) : invalid 'times' argument
## Error in rep(sign.type, remainder) : invalid 'times' argument
## Error in rep(sign.type, remainder) : invalid 'times' argument
## Error in rep(sign.type, remainder) : invalid 'times' argument
## Error in rep(sign.type, remainder) : invalid 'times' argument
## Error in rep(sign.type, remainder) : invalid 'times' argument
## Error in rep(sign.type, remainder) : invalid 'times' argument
## Error in rep(sign.type, remainder) : invalid 'times' argument
## Error in rep(sign.type, remainder) : invalid 'times' argument
## Error in rep(sign.type, remainder) : invalid 'times' argument
## Error in rep(sign.type, remainder) : invalid 'times' argument

```



```
# compile final data frame
```

```
##      tablet.id
## [1,] "P000001"
## [2,] "P000002"
## [3,] "P000008"
## [4,] "P000009"
## [5,] "P000010"
## [6,] "P000011"
##      signs.orig.vec
## [1,] "1(N01) _ 1(N01) _ TIM _ 1(N01) _ 1(N01) _ 1(N01) _ GAL~a# _ 1(N01) _ GAL~a _ UMUN2 _ 1(N01) _ 1(N01) _ GAL~a _ SZITA~a1 _ ABGAL# _ 1(N01) _ GAL~a _ SILA4~b _ 1(N01) _ GAL~a _ SZAB~a"
## [2,] "ISZ~b# NI~b TUR _ 1(N01) _ SAL _ ZATU750"
## [3,] "1(N01) _ GAL~a# _ 1(N01) _ GAL~a# _ 1(N01) _ SANGA~a# _ GURUSZDA# _ 1(N01) _ 1(N01) _ GAL~a _ 1(N01) _ EN~a# _ NUN~a _ 1(N01) _ EN~a _ AMAR _ 1(N01) _ |SZU2.EN~a|# _ 1(N01) _ 1(N01) _ IB~a"
## [4,] "1(N01) _ NAM2# _ DI _ 1(N01) _ NAM2 _ NAM2 _ 1(N01) _ NAM2 _ URU~a1 _ 1(N01) _ NAM2 _ ERIN _ 1(N01)"
##      signs.expand.vec
## [1,] "N01 _ N01 _ TIM _ N01 _ N01 _ N01 _ GAL~a _ N01 _ GAL~a _ UMUN2 _ N01 _ DUB~a _ N01 _ SUG5 _ N01"
## [2,] "GAL~a _ SZITA~a1 _ ABGAL _ N01 _ GAL~a _ SILA4~b _ N01 _ GAL~a _ SZAB~a"
## [3,] "ISZ~b NI~b TUR N01 SAL ZATU750"
```

```
## [4,] "N01 _ GAL~a _ N01 _ GAL~a _ N01 _ SANGA~a _ GURUSZDA _ N01 _ N01 _ GAL~a _ N01 _ N01"
## [5,] "N01 _ EN~a _ NUN~a _ N01 _ EN~a _ AMAR _ N01 _ |SZU2.EN~a| _ N01 _ N01 _ IB~a _ ZATU686~a _ NO
## [6,] "N01 _ NAM2 _ DI _ N01 _ NAM2 _ NAM2 _ N01 _ NAM2 _ URU~a1 _ N01 _ NAM2 _ ERIN _ N01 _ GAL~a _ S
```

Safe to file.

```
write.csv(df, file = paste("~/Github/UpperPalCombinatorics/data/CDLI/",
                           paste(period, "/signs_clean.csv", sep = ""),
                           sep = ""), row.names = F)
```